

AWS Assignment

Auto Scaling, Elastic Load Balancer & Route 53

XYZ Corporation – step-by-step implementation with screenshots

Item	Detail
AWS Region	Asia Pacific (Mumbai) – ap-south-1
Services used	EC2, Launch Templates, Auto Scaling, CloudWatch Alarms, Elastic Load Balancing (ALB), Route 53
Scale-out rule	Add 1 instance when CPU utilization > 80%
Scale-in rule	Remove 1 instance when CPU utilization < 60%
Capacity limits	Minimum 1, Desired 1, Maximum 3 instances

Contents

1. Problem Statement.....	3
Tasks to be performed.....	3
Solution overview.....	3
Resources created.....	3
2. Web Server and Application Load Balancer (Task 2).....	4
Step 1: Launch the EC2 web server.....	4
Step 2: Create the target group and register the instance.....	4
Step 3: Create the Application Load Balancer.....	5
Step 4: Test the application through the load balancer.....	6
3. Auto Scaling Based on CPU Utilization (Task 1).....	7
Step 5: Create the launch template.....	7
Step 6: Create the Auto Scaling group.....	7
Step 7: Create the CloudWatch alarm for CPU above 80%.....	8
Step 8: Create the scale-out policy (add instance).....	9
Step 9: Create the CloudWatch alarm for CPU below 60%.....	9
Step 10: Create the scale-in policy (remove instance).....	10
4. Testing Auto Scaling.....	12
Step 11: Generate CPU load on the instance.....	12
Step 12: Monitor the CloudWatch alarms.....	13
Step 13: Verify scale-out (instances added).....	15
Step 14: Verify scale-in (instances removed).....	17
Step 15: Verify the load balancer target health.....	18
5. Route Traffic to the Company Domain with Route 53 (Task 3).....	19
Step 16: Create a public hosted zone.....	19
Step 17: Create an A (Alias) record to the load balancer.....	19
6. Conclusion.....	20

1. Problem Statement

XYZ Corporation uses on-premise solutions and a limited number of systems. As requests to their application increase, the load also increases, so the corporation has had to buy more systems almost on a regular basis. To cut down on hardware expenses, they decided to move their infrastructure to AWS.

Tasks to be performed

1. Manage the scaling requirements of the company by:
 - a. Deploying multiple compute resources on the cloud as soon as the load increases and CPU utilization exceeds 80%.
 - b. Removing the resources when CPU utilization goes under 60%.
2. Create a load balancer to distribute the load between compute resources.
3. Route the traffic to the company's domain.

Solution overview

The solution uses the following flow:

```

Users --> Route 53 (company domain, A-Alias record)
      --> Application Load Balancer XYZ-ALB (HTTP:80)
      --> Target group XYZ-WEBSERVER-targetgroup
      --> EC2 web servers (Apache) in ap-south-1a / ap-south-1b

CloudWatch alarms XYZ-CPU-Above-80 / XYZ-CPU-Below-60
      --> Auto Scaling group XYZ-ASG (min 1, max 3)
      --> Step scaling policies XYZ-ScaleOut / XYZ-ScaleIn
  
```

Resources created

Resource	Name / ID	Key settings
EC2 web server	XYZ-WebServer (i-013b522d00713da44)	t3.micro, Apache httpd, ap-south-1b
Target group	XYZ-WEBSERVER-targetgroup	Target type Instance, HTTP:80, HTTP1
Load balancer	XYZ-ALB	Application, Internet-facing, ap-south-1a & ap-south-1b
ALB security group	XYZ-ALB-SG (sg-0f64778dc9c20d7d3)	2 inbound rules, 1 outbound rule
Launch template	XYZ-LaunchTemplate (lt-0e571ae0ed931caac)	AMI ami-08188a5a4dfdbd573, t3.micro, key pair ELB&53
Auto Scaling group	XYZ-ASG	Desired 1, Min 1, Max 3, 2 Availability Zones
CloudWatch alarm	XYZ-CPU-Above-80	CPUUtilization > 80 for 1 datapoint within 5 minutes
CloudWatch alarm	XYZ-CPU-Below-60	CPUUtilization < 60 for 1 datapoint within 5 minutes
Scaling policy	XYZ-ScaleOut	Step scaling: add 1 capacity unit, 300 s warm-up
Scaling policy	XYZ-ScaleIn	Step scaling: remove 1 capacity unit
Route 53	Hosted zone + A (Alias) record	Alias target: XYZ-ALB

2. Web Server and Application Load Balancer (Task 2)

This part creates the web server, registers it with a target group, and puts an internet-facing Application Load Balancer (ALB) in front of it.

Step 1: Launch the EC2 web server

1. Open the **EC2** console in region **Asia Pacific (Mumbai)** and click **Launch instances**.
2. Name: **XYZ-WebServer**. AMI: Amazon Linux. Instance type: **t3.micro**.
3. Select a key pair (for example **ELB&53**) and a security group that allows **HTTP (80)** and **SSH (22)**.
4. Under **Advanced details** → **User data**, add a script to install Apache and publish a test page (sample below).
5. Click **Launch instance** and wait until the instance shows **Running** with **3/3 checks passed**.

Sample user data:

```
#!/bin/bash
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
echo "<h1>XYZ Corporation Web Server</h1>
<h2>Server: $(hostname -f)</h2>
<p>Welcome to XYZ Corporation AWS Application</p>" > /var/www/html/index.html
```

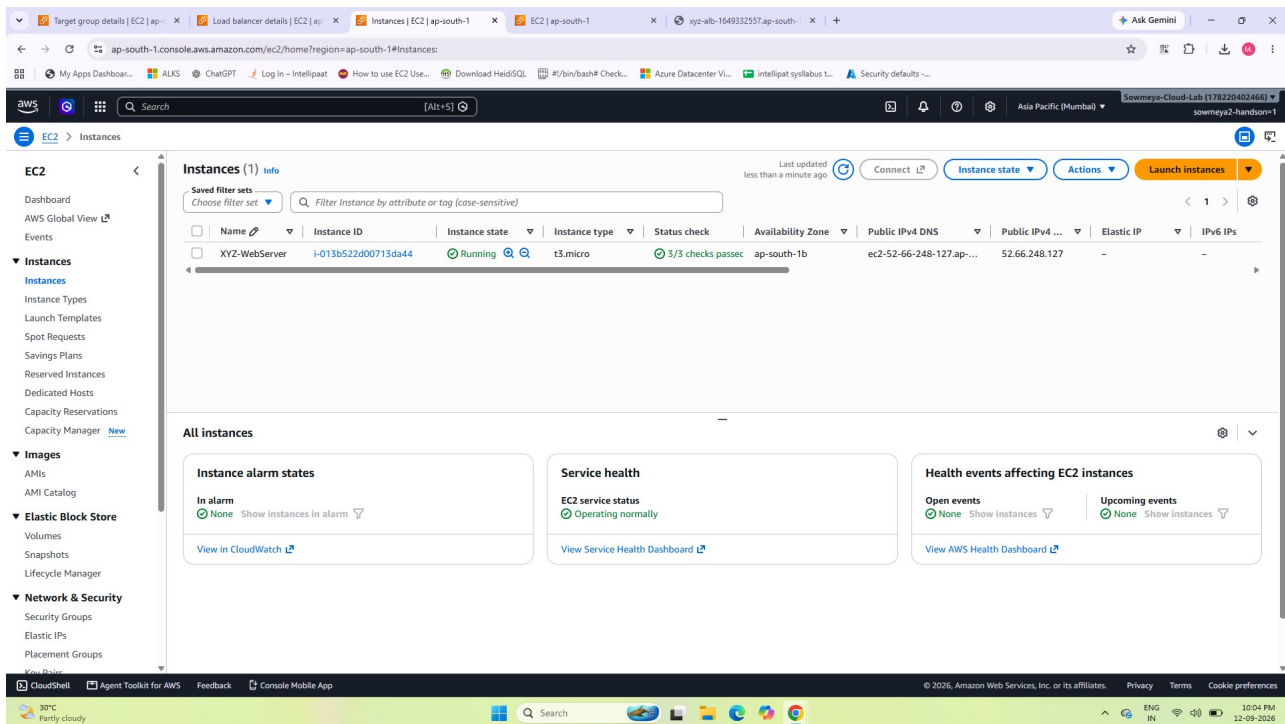


Figure 1: XYZ-WebServer instance running (t3.micro, ap-south-1b, 3/3 status checks passed)

Step 2: Create the target group and register the instance

1. In the EC2 console, go to **Load Balancing** → **Target Groups** → **Create target group**.
2. Target type: **Instances**. Name: **XYZ-WEBSEVER-targetgroup**. Protocol/Port: **HTTP : 80**. Protocol version: **HTTP1**.
3. Select the default VPC and keep the default health check path (/).

- Click **Next**, select **XYZ-WebServer**, click **Include as pending below**, then **Create target group**.

The target group confirms one target registered successfully and the target is reported Healthy.

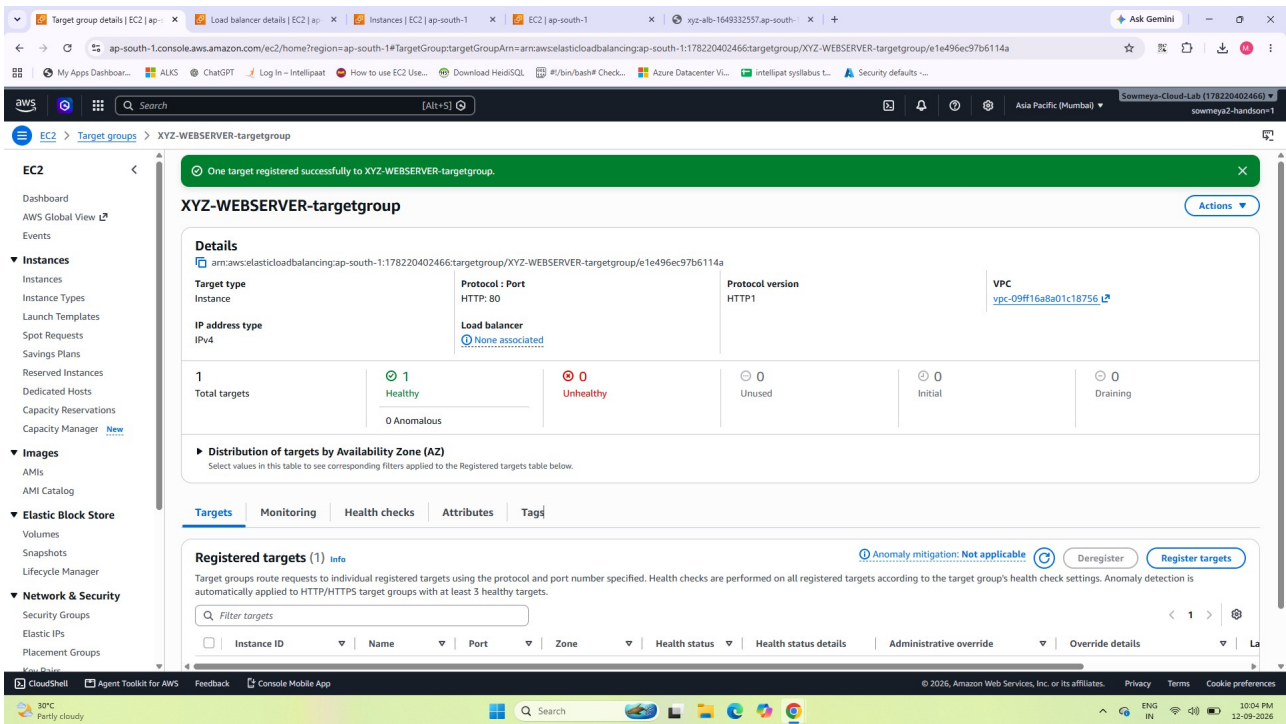


Figure 2: Target group XYZ-WEBSEVER-targetgroup created with 1 healthy target

Step 3: Create the Application Load Balancer

- Go to **Load Balancers** → **Create load balancer** → **Application Load Balancer**.
- Name: **XYZ-ALB**. Scheme: **Internet-facing**. IP address type: **IPv4**.
- Network mapping: select the VPC and two Availability Zones – **ap-south-1a** and **ap-south-1b**.
- Security group: create/select **XYZ-ALB-SG** allowing inbound HTTP (80) from anywhere.
- Listener: **HTTP : 80** → forward to **XYZ-WEBSEVER-targetgroup**. Click **Create load balancer**.
- Wait until the status is **Active**, then copy the **DNS name**.

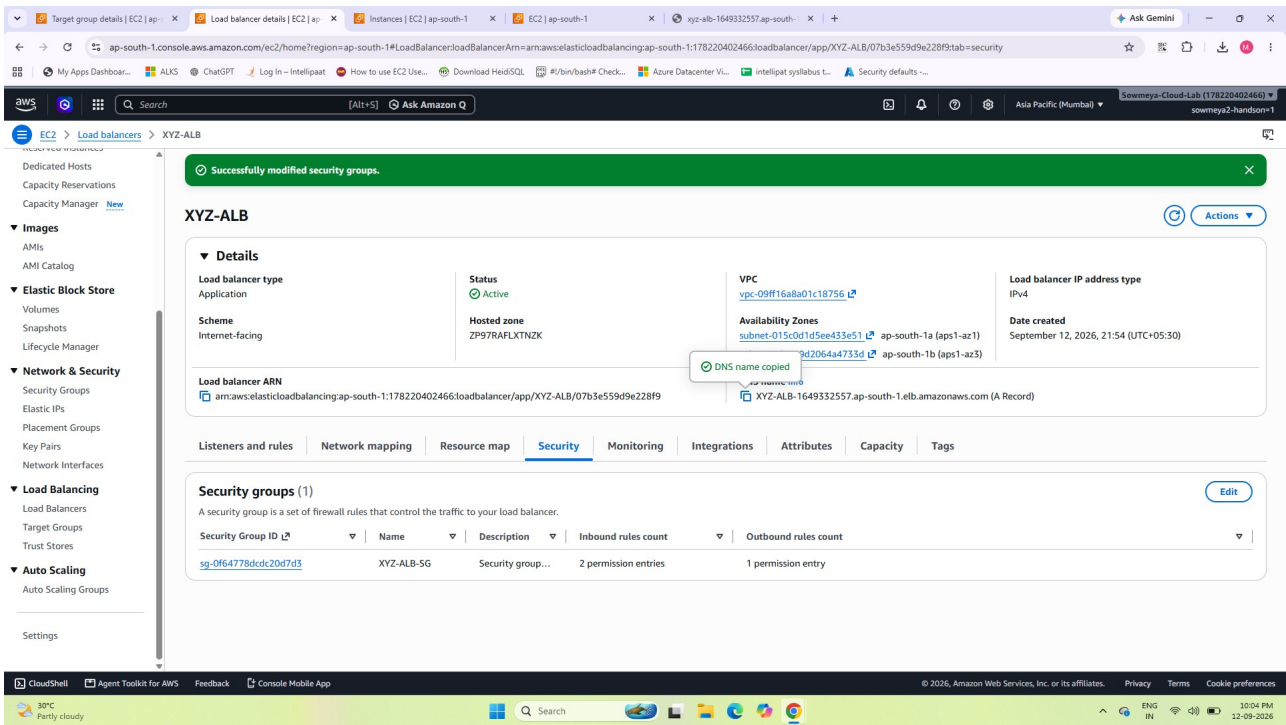


Figure 3: XYZ-ALB is Active (Application, Internet-facing) with security group XYZ-ALB-SG; DNS name copied

Note: Record the ALB DNS name (XYZ-ALB-1649332557.ap-south-1.elb.amazonaws.com) and Hosted zone (ZP97RAFLXTNZK). These are used later for the Route 53 alias record.

Step 4: Test the application through the load balancer

Paste the ALB DNS name in a browser: <http://xyz-alb-1649332557.ap-south-1.elb.amazonaws.com>. The page served by the web server confirms traffic is flowing through the load balancer.

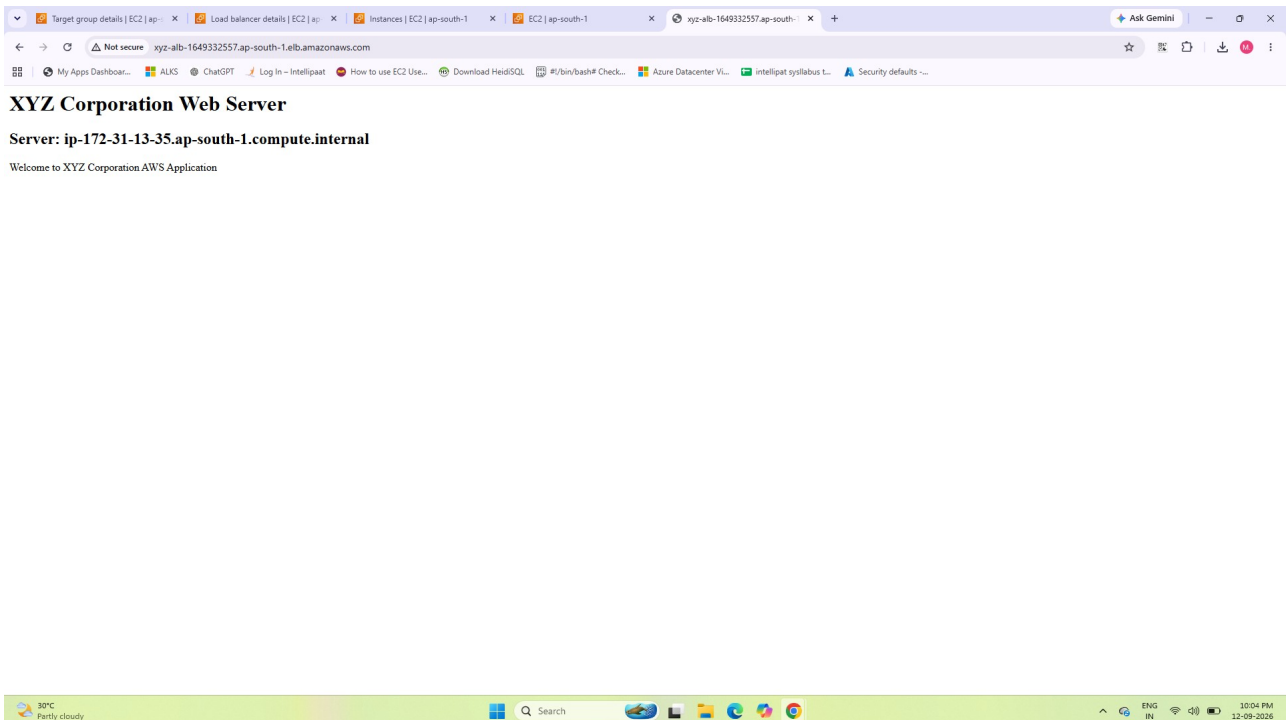


Figure 4: XYZ Corporation Web Server page served through the ALB DNS name

3. Auto Scaling Based on CPU Utilization (Task 1)

This part creates a launch template and an Auto Scaling group, then adds two CloudWatch alarms and two step scaling policies: one to add an instance when CPU goes above 80%, and one to remove an instance when CPU goes below 60%.

Step 5: Create the launch template

1. In the EC2 console, go to **Instances → Launch Templates → Create launch template**.
2. Name: **XYZ-LaunchTemplate**. Description: **Launch template for XYZ Corporation Auto Scaling**.
3. AMI: **ami-08188a5a4dfdbd573**. Instance type: **t3.micro**. Key pair: **ELB&53**.
4. Security group: **sg-0ff16f29804c167f8** (allows HTTP and SSH).
5. Add the same Apache **user data** as the web server, then click **Create launch template**.

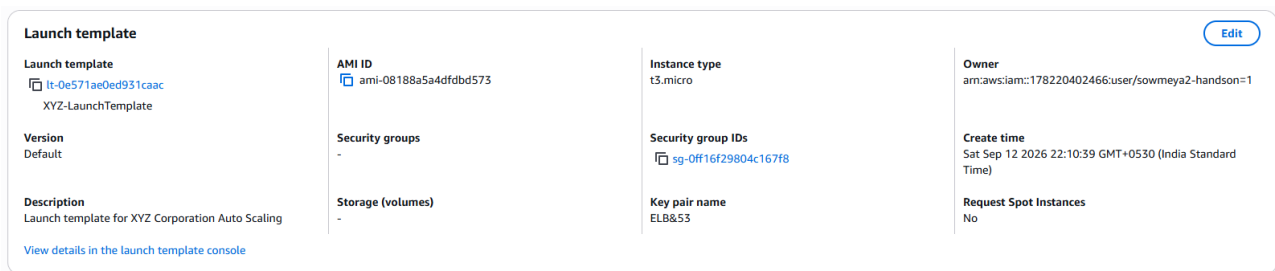


Figure 5: Launch template XYZ-LaunchTemplate details (AMI, t3.micro, security group, key pair ELB&53)

Step 6: Create the Auto Scaling group

1. Go to **Auto Scaling → Auto Scaling Groups → Create Auto Scaling group**.
2. Name: **XYZ-ASG**. Launch template: **XYZ-LaunchTemplate** (Version Default).
3. Network: select the VPC and subnets in **2 Availability Zones** (ap-south-1a and ap-south-1b).
4. Group size: **Desired 1, Minimum 1, Maximum 3**.
5. Scaling policies: choose **No scaling policies** for now (step policies are added after the alarms). Click **Create Auto Scaling group**.

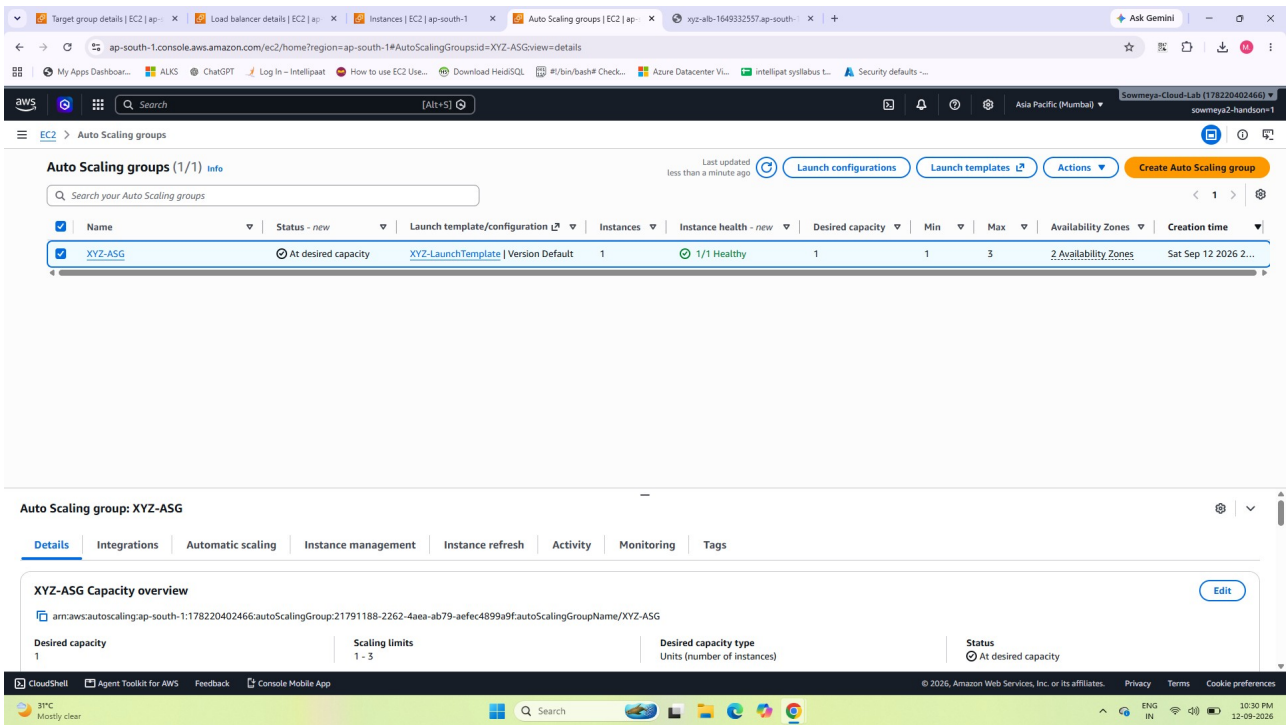


Figure 6: XYZ-ASG created – At desired capacity, 1 instance, Min 1 / Max 3, 2 Availability Zones

Step 7: Create the CloudWatch alarm for CPU above 80%

1. Open **CloudWatch** → **Alarms** → **Create alarm** → **Select metric**.
2. Choose **EC2** → **Per-Instance Metrics** → **CPUUtilization** for the web server instance (i-013b522d00713da44).
3. Statistic: **Average**, Period: **5 minutes**. Condition: **Greater than 80**, 1 out of 1 datapoints.
4. Skip notifications (the scaling action is attached from the Auto Scaling side). Name: **XYZ-CPU-Above-80**. Click **Create alarm**.

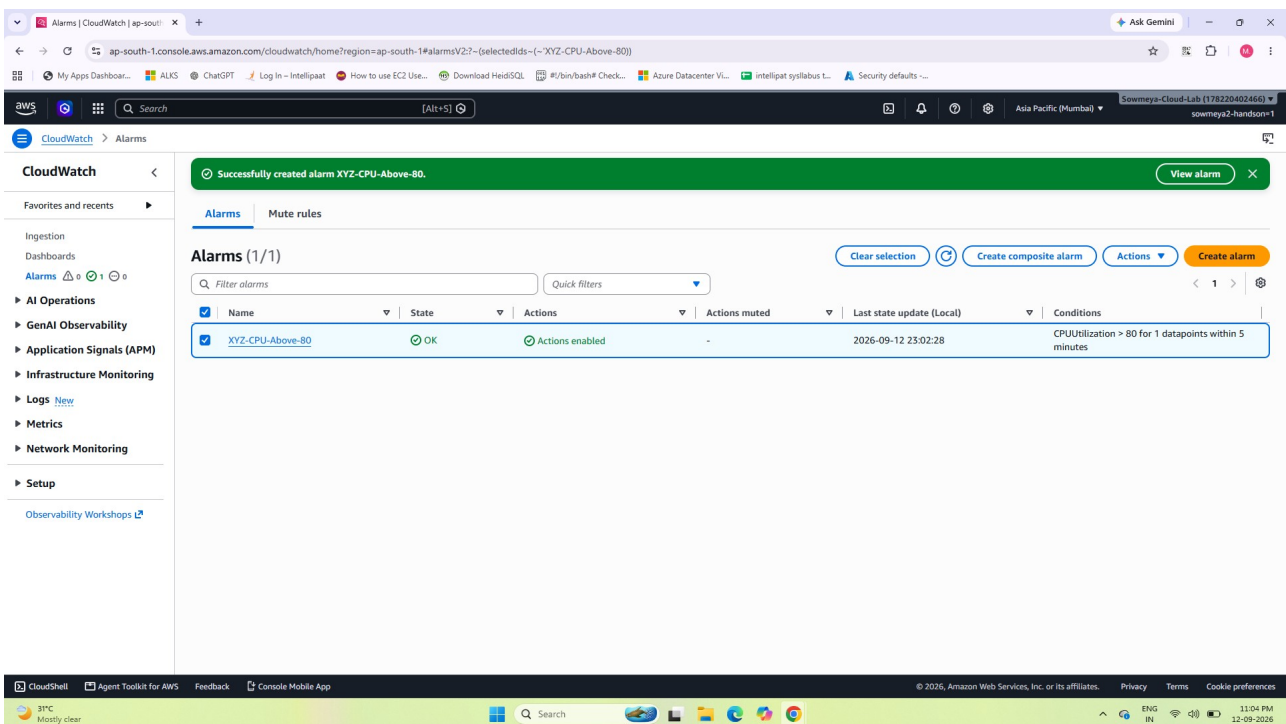


Figure 7: Alarm XYZ-CPU-Above-80 created (CPUUtilization > 80 for 1 datapoint within 5 minutes)

Step 8: Create the scale-out policy (add instance)

1. Open **XYZ-ASG** → **Automatic scaling** → **Create dynamic scaling policy**.
2. Policy type: **Step scaling**. Name: **XYZ-ScaleOut**.
3. CloudWatch alarm: **XYZ-CPU-Above-80**.
4. Take the action: **Add 1 capacity units** when $80 \leq \text{CPUUtilization} < +\infty$.
5. Instances need **300 seconds** to warm up. Click **Create**.

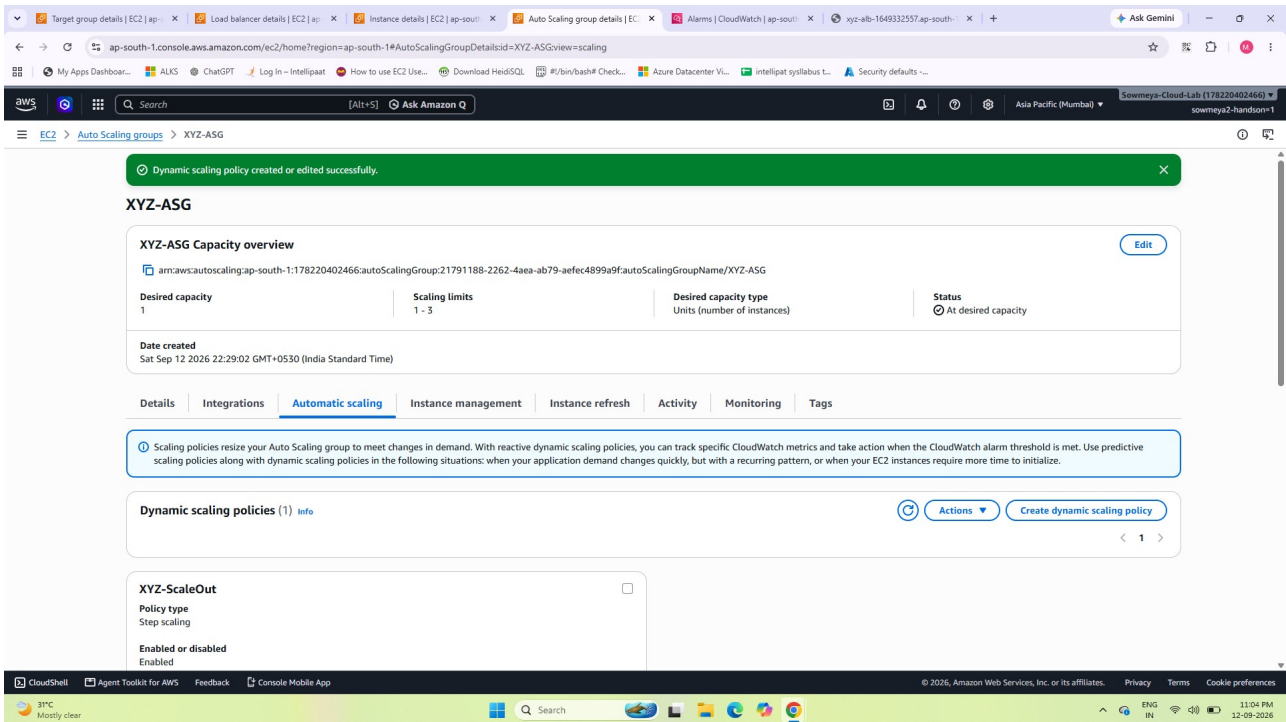


Figure 8: Dynamic scaling policy XYZ-ScaleOut (Step scaling, Enabled) created successfully

Step 9: Create the CloudWatch alarm for CPU below 60%

1. In CloudWatch, click **Create alarm** and select the same **CPUUtilization** metric.
2. Condition: **Lower than 60**, period 5 minutes, 1 out of 1 datapoints.
3. Name: **XYZ-CPU-Below-60**. Click **Create alarm**.

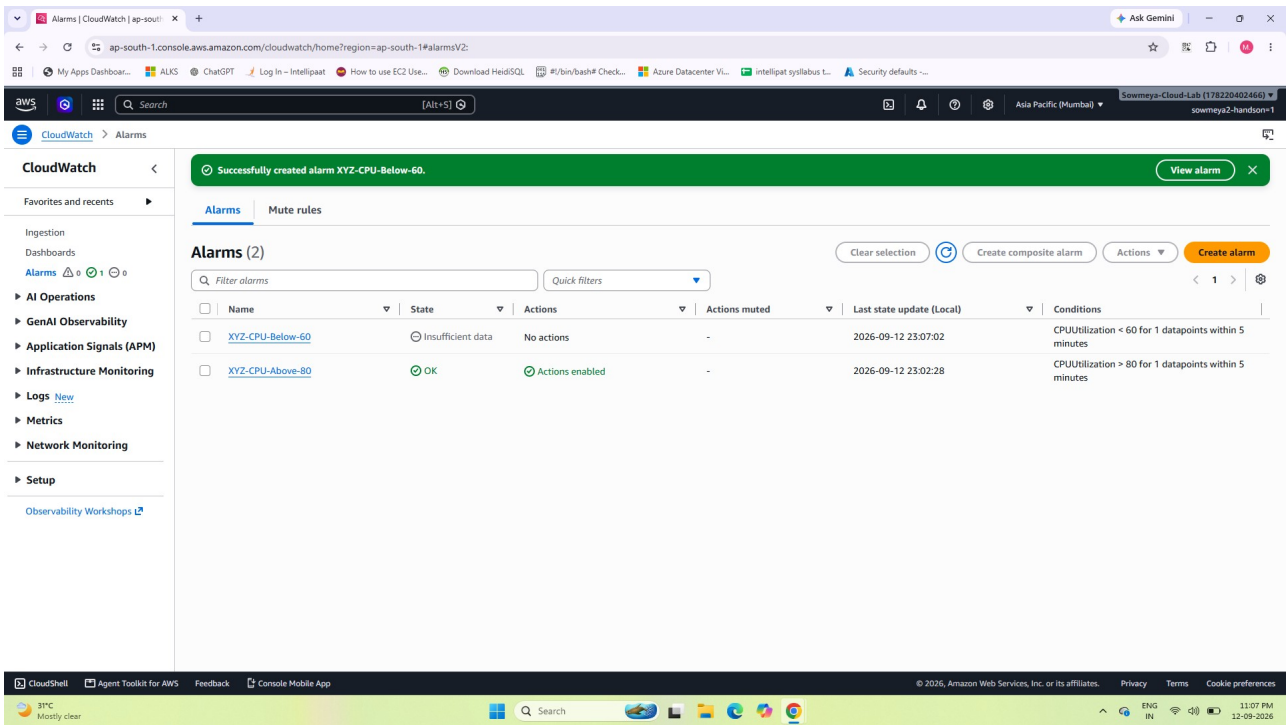


Figure 9: Alarm XYZ-CPU-Below-60 created alongside XYZ-CPU-Above-80

Step 10: Create the scale-in policy (remove instance)

1. In XYZ-ASG → Automatic scaling, click **Create dynamic scaling policy**.
2. Policy type: **Step scaling**. Name: **XYZ-ScaleIn**. CloudWatch alarm: **XYZ-CPU-Below-60**.
3. Take the action: **Remove 1 capacity units** when $60 \geq \text{CPUUtilization} > -\text{infinity}$. Click **Create**.

Both dynamic scaling policies now appear on the Auto Scaling group:

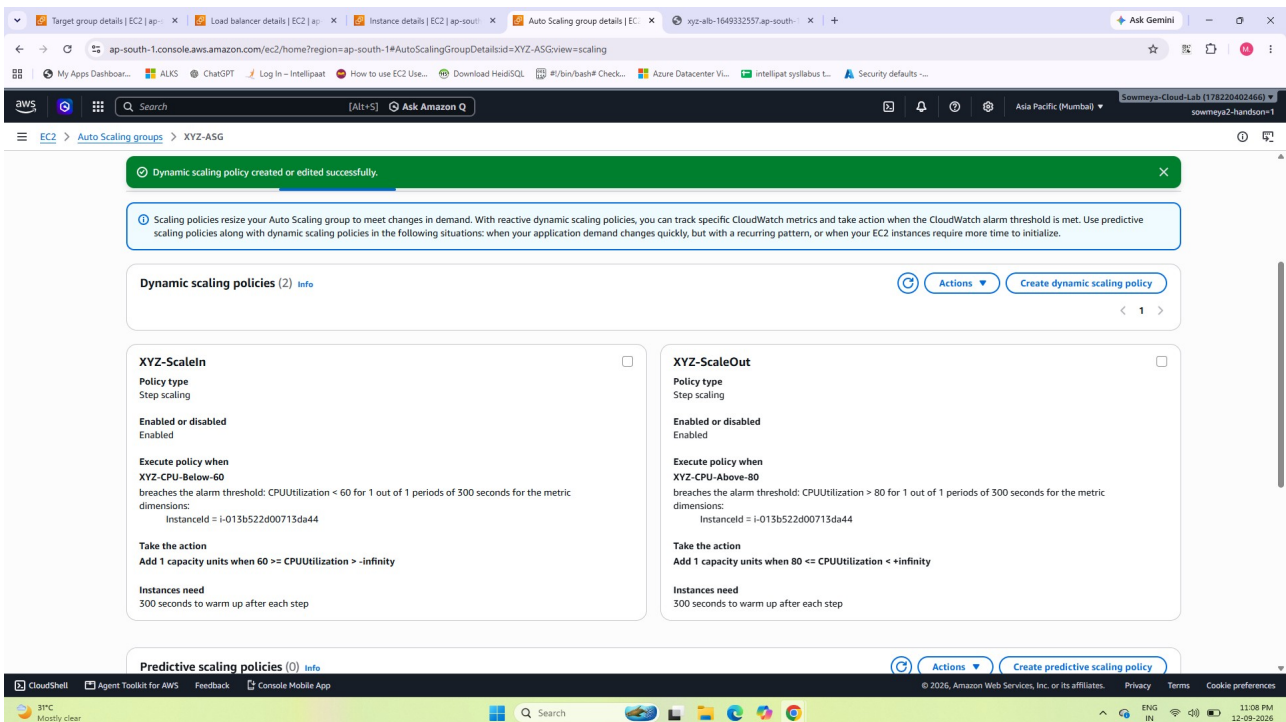


Figure 10: Dynamic scaling policies XYZ-ScaleIn and XYZ-ScaleOut listed on XYZ-ASG

Check: Make sure the scale-in action is set to **Remove**, not **Add**. The policy was edited (Actions → Edit) so the final configuration reads **Remove 1 capacity units when 60 >= CPUUtilization > -infinity**, as shown below.

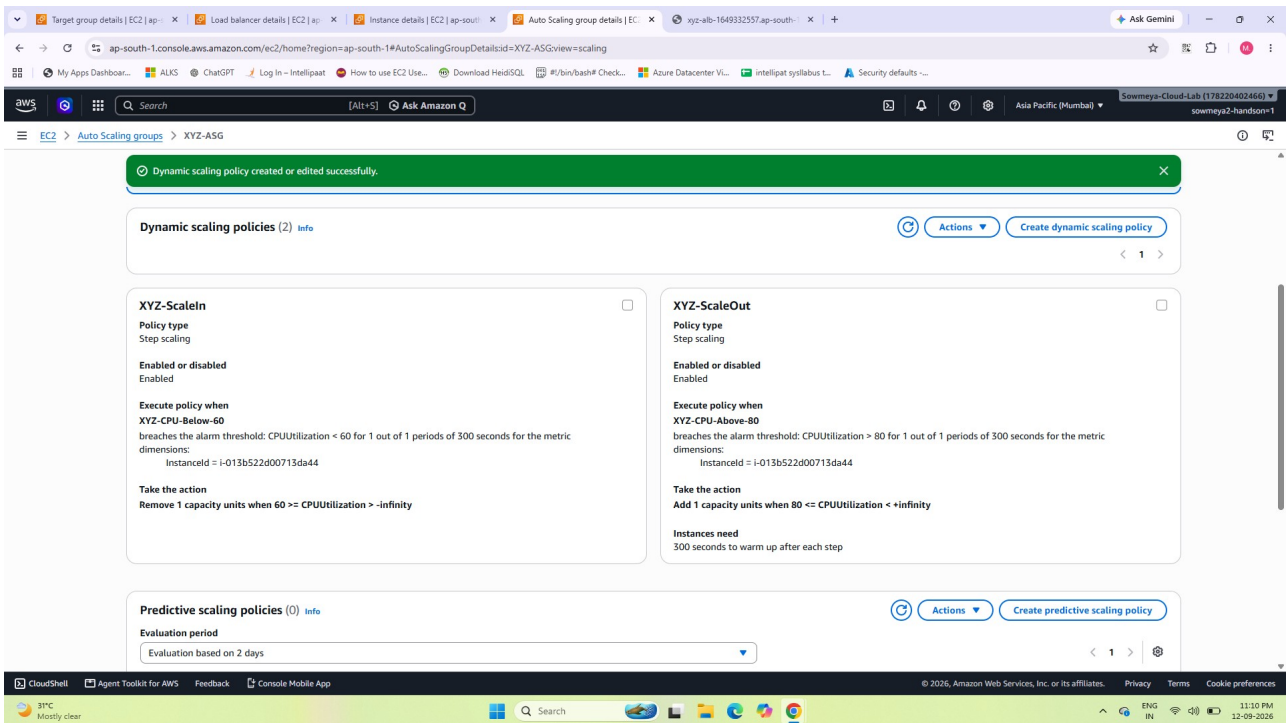


Figure 11: Final configuration – XYZ-ScaleIn removes 1 instance; XYZ-ScaleOut adds 1 instance

4. Testing Auto Scaling

To prove the scaling rules work, CPU load was generated on the web server and the alarms, instance count and Auto Scaling group capacity were observed.

Step 11: Generate CPU load on the instance

1. Select **XYZ-WebServer** → **Connect** → **EC2 Instance Connect** → **Connect**.
2. Start a CPU-intensive process and monitor it with top:

```
yes > /dev/null &      # start CPU load (run again to add more load)
top                    # watch %CPU of the 'yes' process
kill yes               # stop the load when testing is finished
```

The **yes** process uses close to 100% CPU on the instance.

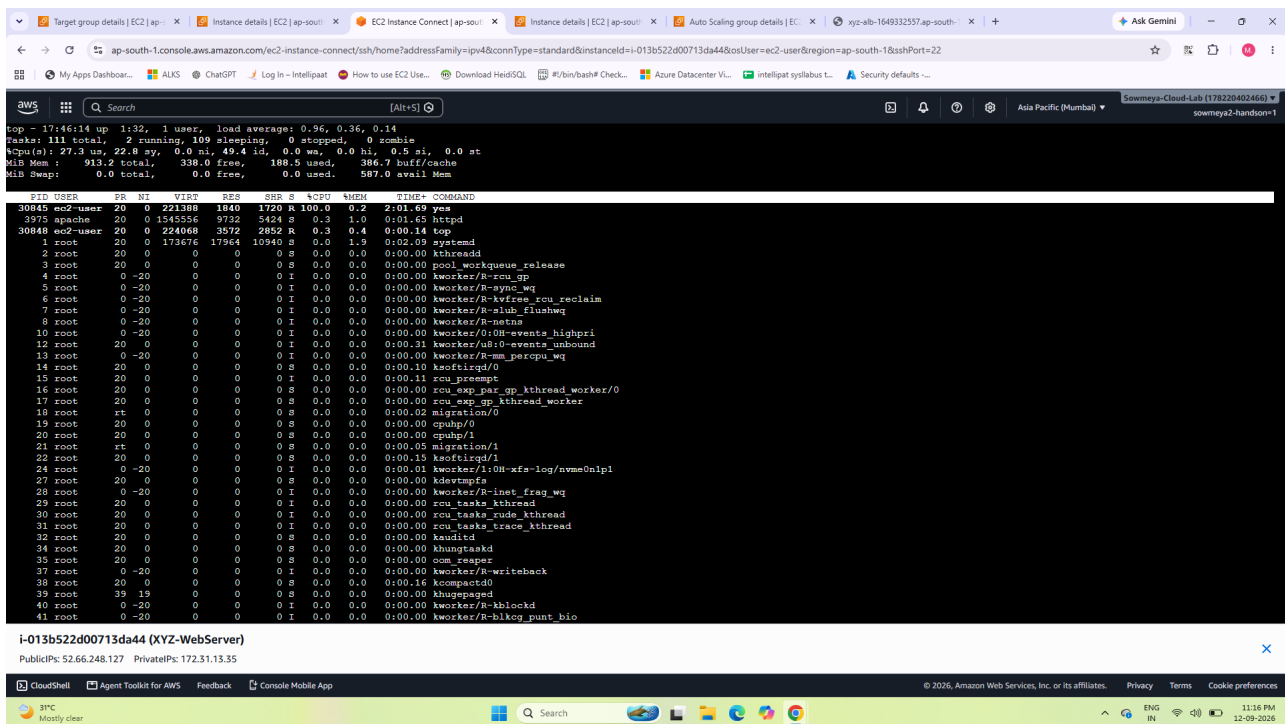


Figure 12: top output – the 'yes' process running at 100% CPU

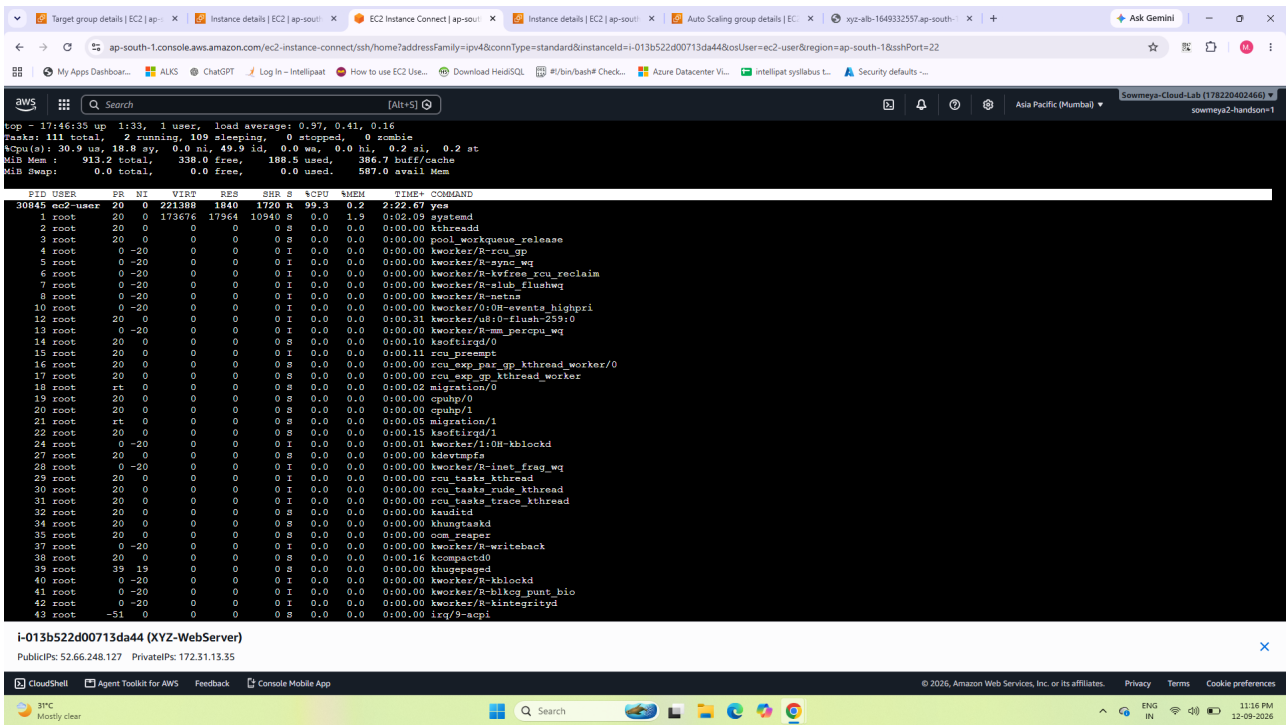


Figure 13: Sustained CPU load – 'yes' process at 99.3% CPU

Step 12: Monitor the CloudWatch alarms

Before the load builds up, CPU is low, so XYZ-CPU-Below-60 is In alarm and XYZ-CPU-Above-80 is OK:

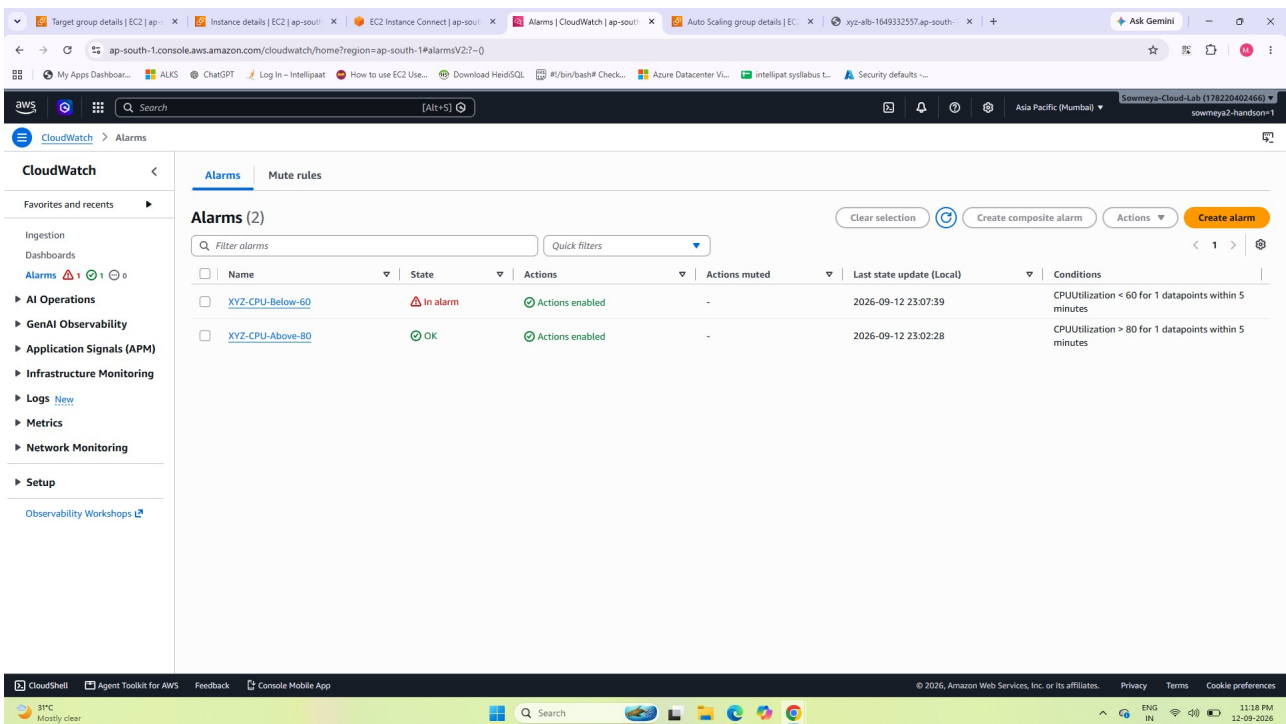


Figure 14: Low CPU – XYZ-CPU-Below-60 In alarm, XYZ-CPU-Above-80 OK

Once CPU crosses 80%, the states flip: XYZ-CPU-Above-80 goes In alarm, which triggers the XYZ-ScaleOut policy.

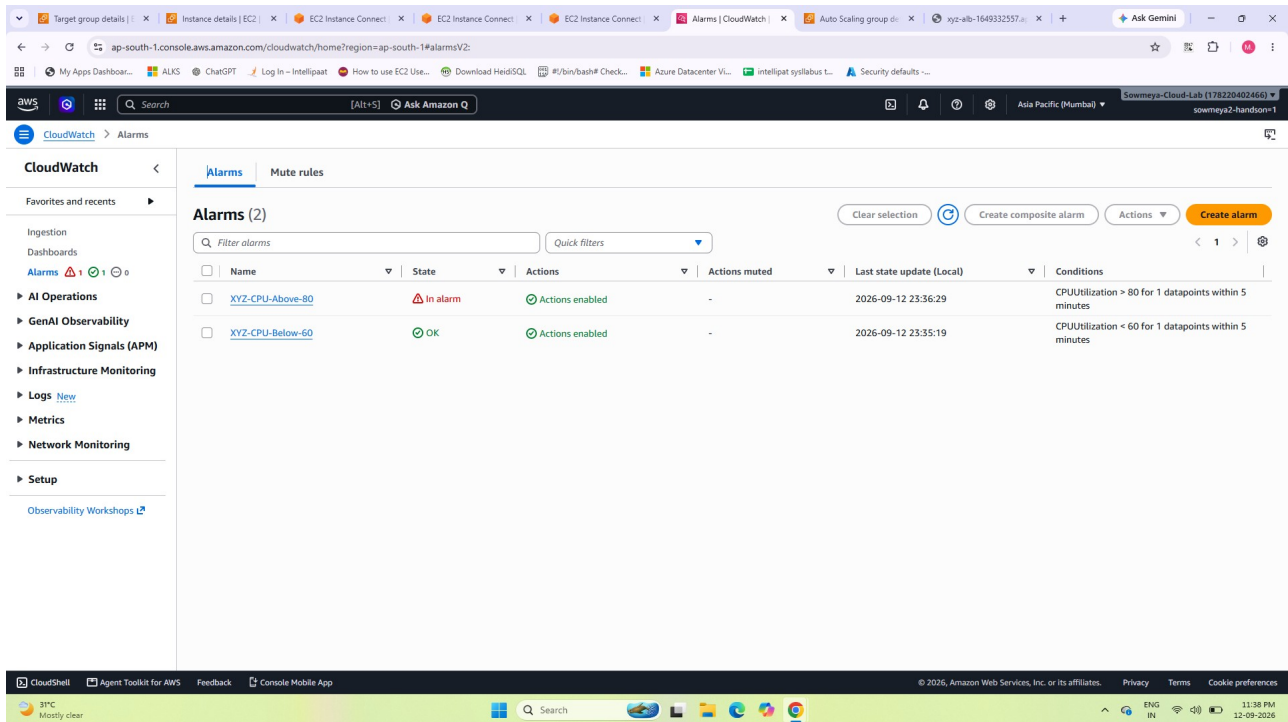


Figure 15: High CPU – XYZ-CPU-Above-80 In alarm, XYZ-CPU-Below-60 OK

The alarm graph shows CPUUtilization rising above the 80% threshold line:

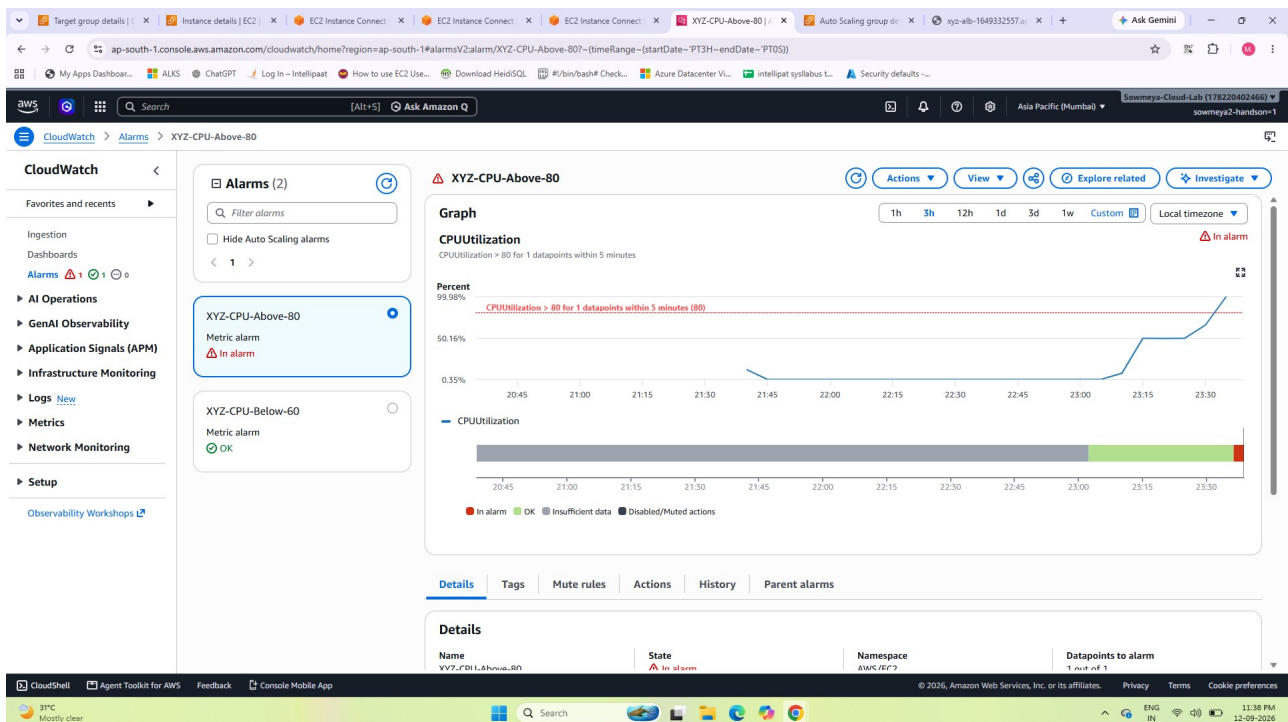


Figure 16: XYZ-CPU-Above-80 graph – CPUUtilization crosses the 80% threshold

Step 13: Verify scale-out (instances added)

The Auto Scaling group launches new t3.micro instances across ap-south-1a and ap-south-1b, and its desired capacity increases to 2.

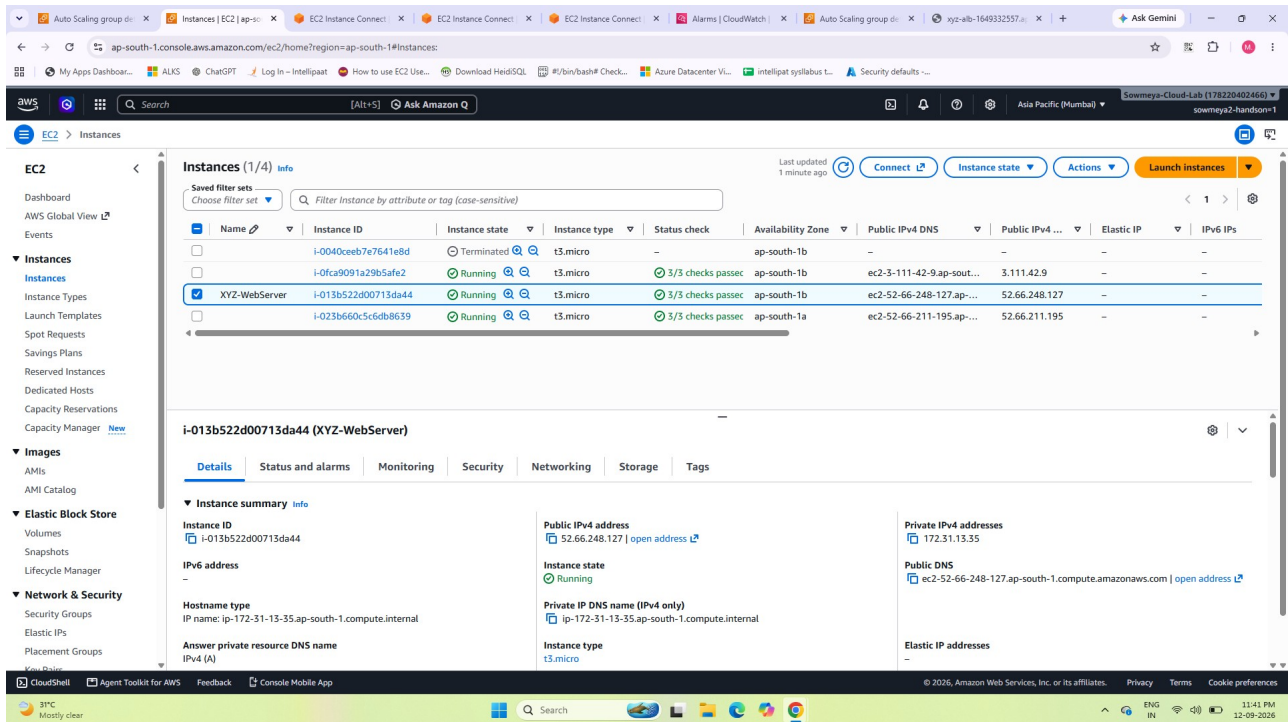


Figure 17: EC2 Instances – new Auto Scaling instances launched and running

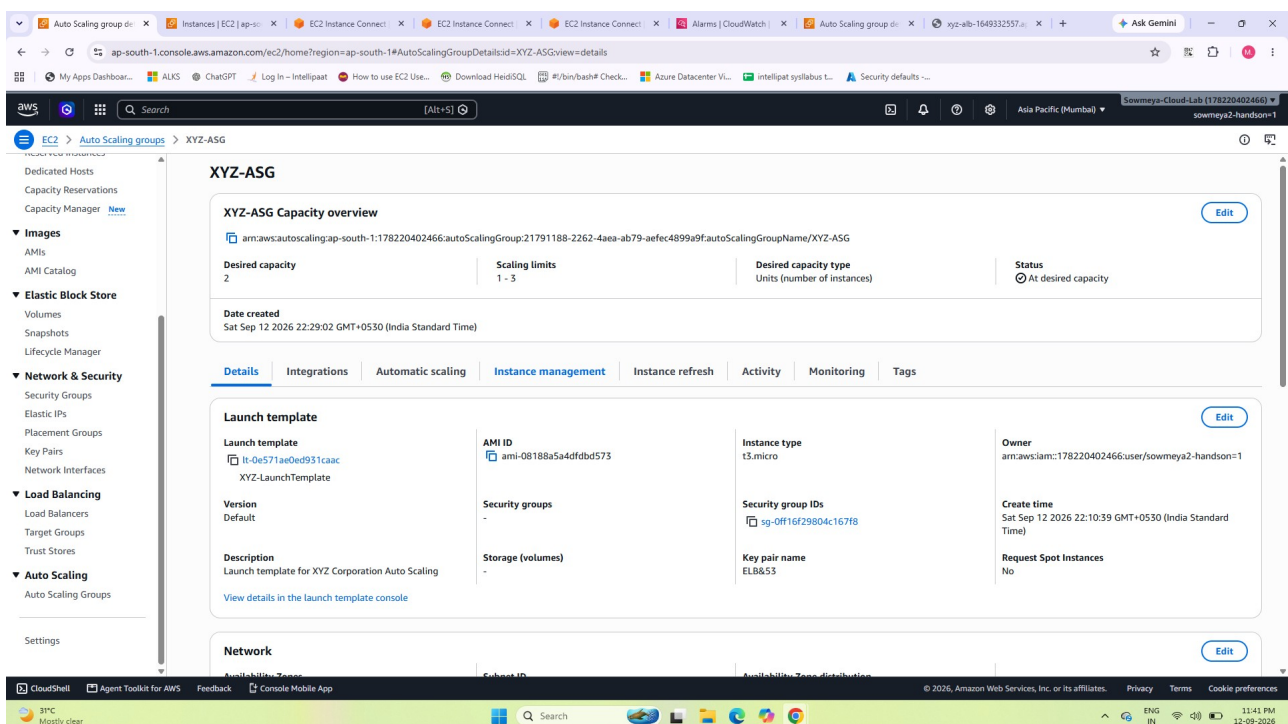


Figure 18: XYZ-ASG desired capacity increased to 2 (scaling limits 1 – 3)

Additional load was added by starting more yes processes:

The screenshot shows a terminal window with the following output from the 'top' command:

```
top - 10:12:21 up 1:50, 3 users, load averages: 3.07, 2.58, 1.69
tasks: 119 total, 4 running, 115 sleeping, 0 stopped, 0 zombie
%Cpu(s): 64.2 us, 35.8 sy, 0.0 ni, 0.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 913.2 total, 311.1 free, 214.9 used, 387.2 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used, 560.5 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	MEM	TIME+	COMMAND
30413	ec2-user	20	0	221388	1840	1720	R	70.8	0.2	25:54.63	yes
32100	ec2-user	20	0	221388	1836	1716	R	69.3	0.2	7:12.42	yes
32649	ec2-user	20	0	221388	1840	1716	R	59.7	0.2	3:35.20	yes
30840	ec2-user	20	0	224068	3580	2852	R	0.7	0.4	0:02.84	top
32121	ec2-user	20	0	224068	3584	2848	S	0.2	0.4	0:00.95	top
1	root	20	0	173676	17968	10940	S	0.0	1.9	0:02.43	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:00.00	pool_workqueue_release
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-rcu_gp
5	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-sync_wq
6	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-krefm_rcu_reclaim
7	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-mlub_flushwq
8	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-netns
10	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/0:0H-events_highpri
13	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-mm_percpu_wq
14	root	20	0	0	0	0	S	0.0	0.0	0:00.13	ksoftirqd/0
15	root	20	0	0	0	0	I	0.0	0.0	0:00.15	rcu_preempt
16	root	20	0	0	0	0	S	0.0	0.0	0:00.00	rcu_exp_par_gp_kthread_worker/0
17	root	20	0	0	0	0	S	0.0	0.0	0:00.00	rcu_exp_gp_kthread_worker
18	root	rt	0	0	0	0	S	0.0	0.0	0:00.04	migration/0
19	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/0
20	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/1
21	root	rt	0	0	0	0	S	0.0	0.0	0:00.06	migration/1
22	root	20	0	0	0	0	S	0.0	0.0	0:00.16	ksoftirqd/1
24	root	0	-20	0	0	0	I	0.0	0.0	0:00.01	kworker/1:0H-kblockd
27	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kdevtmpfs
28	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-inet_frag_wq
29	root	20	0	0	0	0	S	0.0	0.0	0:00.00	rcu_tasks_kthread
30	root	20	0	0	0	0	I	0.0	0.0	0:00.00	rcu_tasks_rude_kthread
31	root	20	0	0	0	0	I	0.0	0.0	0:00.00	rcu_tasks_trace_kthread
32	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kauditd
34	root	20	0	0	0	0	S	0.0	0.0	0:00.00	khungtaskd
35	root	20	0	0	0	0	S	0.0	0.0	0:00.00	oom_reaper
37	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-writeback
38	root	20	0	0	0	0	S	0.0	0.0	0:00.15	kcompactd0
39	root	39	19	0	0	0	S	0.0	0.0	0:00.00	khugepaged
40	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-kblockd

Below the terminal output, the AWS console shows the instance details for **i-013b522d00713da44 (XYZ-WebServer)**, including its Public IP (52.66.248.127) and Private IP (172.31.13.35).

Figure 19: Multiple 'yes' processes running to keep CPU utilization high

With CPU still above 80%, another instance is launched (status: Initializing):

The screenshot shows the AWS Management Console 'Instances' page. The table lists the following instances:

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
	i-0040ceeb7e7641e8d	Terminated	t3.micro	-	ap-south-1b	-	-	-	-
	i-0dbb4e990c47ae07	Running	t3.micro	-	ap-south-1b	ec2-15-206-90-131.ap...	15.206.90.131	-	-
	i-0fca9091a29b5afe2	Running	t3.micro	3/3 checks passed	ap-south-1b	ec2-3-111-42-9.ap-sout...	3.111.42.9	-	-
XYZ-WebServer	i-013b522d00713da44	Running	t3.micro	3/3 checks passed	ap-south-1b	ec2-52-66-248-127.ap...	52.66.248.127	-	-
	i-023b660c5c6db8639	Running	t3.micro	3/3 checks passed	ap-south-1a	ec2-52-66-211-195.ap...	52.66.211.195	-	-

Below the table, the details for the selected instance **i-013b522d00713da44 (XYZ-WebServer)** are shown, including its Instance ID, IP addresses, and Instance type (t3.micro).

Figure 20: Another instance launched by Auto Scaling (Initializing)

Step 14: Verify scale-in (instances removed)

Stop the load with **pkill yes**. When CPU drops below 60%, XYZ-CPU-Below-60 goes In alarm and XYZ-CPU-Above-80 returns to OK. This triggers the XYZ-ScaleIn policy.

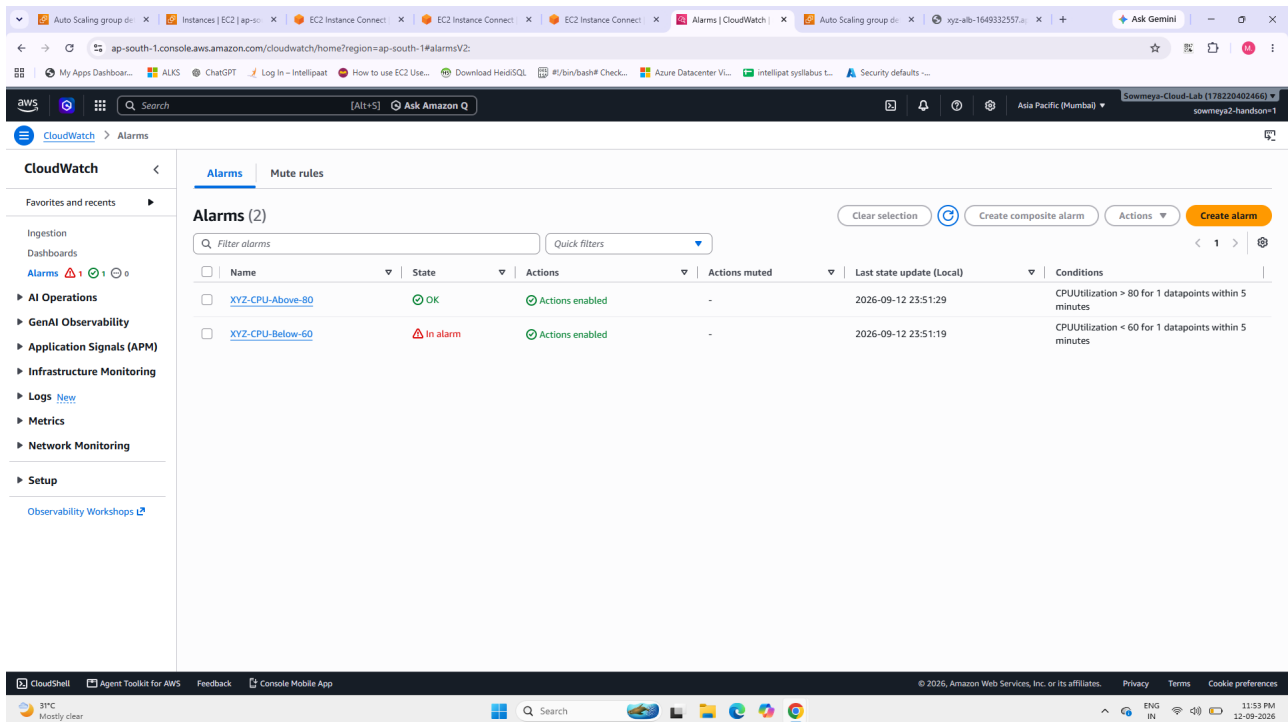


Figure 21: CPU below 60% – XYZ-CPU-Below-60 In alarm, XYZ-CPU-Above-80 OK

The Auto Scaling group removes one instance and the desired capacity comes back down to 2. It keeps scaling in, one instance at a time, while CPU stays below 60%, down to the minimum of 1.

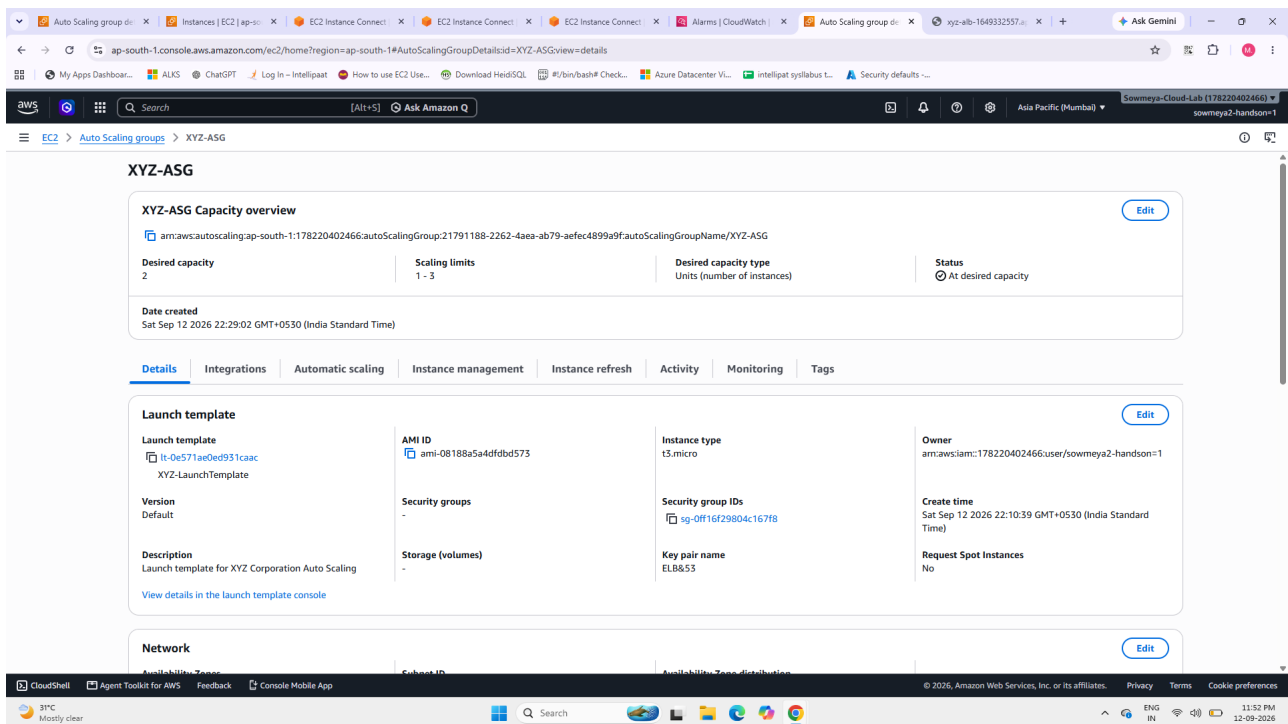


Figure 22: XYZ-ASG after scale-in – desired capacity reduced to 2

Step 15: Verify the load balancer target health

The target group is now associated with XYZ-ALB and the registered web server target is Healthy, so the load balancer is distributing traffic to it.

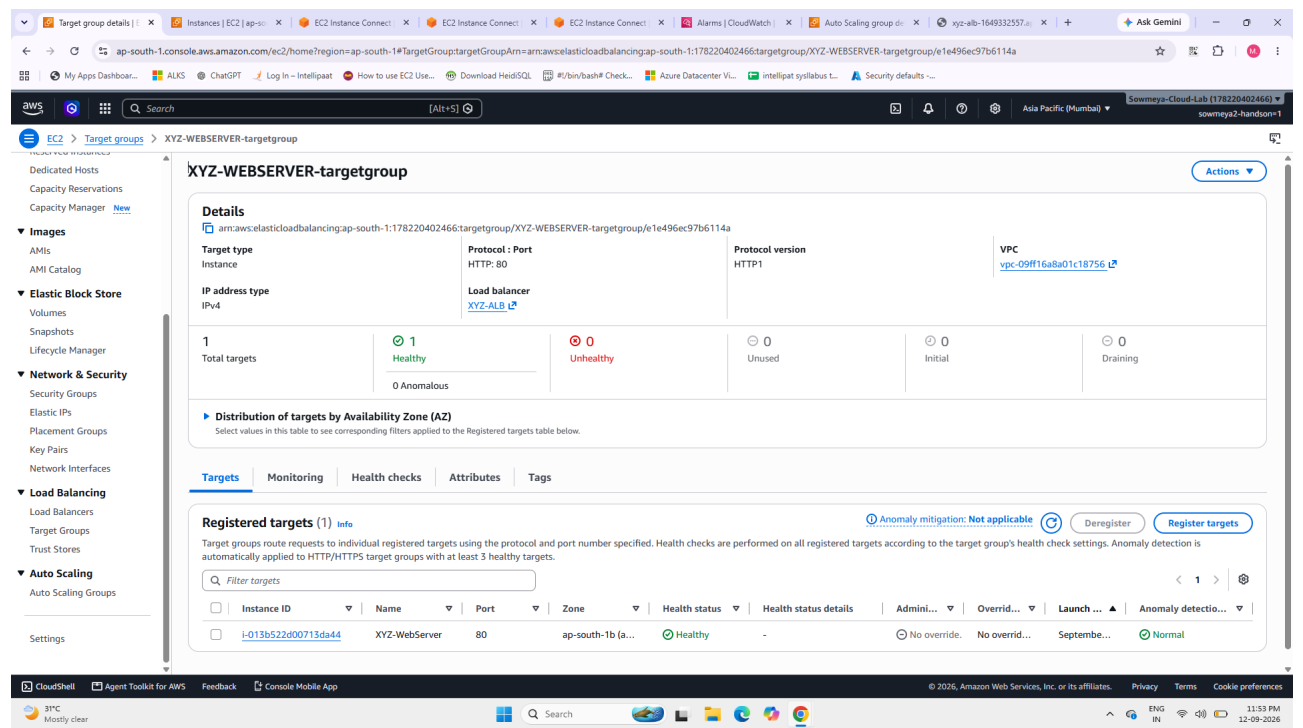


Figure 23: XYZ-WEBSEVER-targetgroup associated with XYZ-ALB – target Healthy

5. Route Traffic to the Company Domain with Route 53 (Task 3)

The final step points the company's domain to the Application Load Balancer using a Route 53 alias record. In the steps below, xyzcorp.com is used as an example domain name.

Step 16: Create a public hosted zone

1. Open the **Route 53** console → **Hosted zones** → **Create hosted zone**.
2. Domain name: **xyzcorp.com**. Type: **Public hosted zone**. Click **Create hosted zone**.
3. Copy the four **NS** (name server) values and update them at the domain registrar (not needed if the domain was registered in Route 53).

Step 17: Create an A (Alias) record to the load balancer

1. Open the hosted zone and click **Create record**.
2. Record name: **www** (or leave blank for the root domain). Record type: **A – Routes traffic to an IPv4 address and some AWS resources**.
3. Turn on **Alias**. Route traffic to: **Alias to Application and Classic Load Balancer**.
4. Region: **Asia Pacific (Mumbai)**. Load balancer: **dualstack.xyz-alb-1649332557.ap-south-1.elb.amazonaws.com**.
5. Routing policy: **Simple routing**. Evaluate target health: **Yes**. Click **Create records**.

6. Conclusion

All three tasks were implemented on AWS in the ap-south-1 region:

Requirement	How it was met	Evidence
1a. Add compute when CPU > 80%	CloudWatch alarm XYZ-CPU-Above-80 + step scaling policy XYZ-ScaleOut (add 1 instance) on XYZ-ASG	Figures 7, 8, 12-20
1b. Remove compute when CPU < 60%	CloudWatch alarm XYZ-CPU-Below-60 + step scaling policy XYZ-ScaleIn (remove 1 instance)	Figures 9-11, 21, 22
2. Load balancer	Internet-facing ALB XYZ-ALB across 2 AZs, forwarding HTTP:80 to XYZ-WEBSERVER-targetgroup	Figures 1-4, 23
3. Route traffic to domain	Route 53 public hosted zone with an A (Alias) record pointing to XYZ-ALB	Section 5

Moving to Auto Scaling means XYZ Corporation no longer needs to buy hardware ahead of demand: capacity is added automatically when load rises and removed when it falls, so they pay only for what is used.